

Documentation Technique

Plateforme JobAI

Prédiction de salaire et gestion d'offres d'emploi

Rapport structuré avec architecture, commandes, flux
métier et extraits de code importants

Réalisé par
Farah Soltani

Table des matières

1	Présentation générale	3
2	Objectifs fonctionnels	3
3	Diagramme d'architecture	3
3.1	Lecture du flux	3
4	Tableau des commandes	4
5	Structure logique du projet	4
5.1	Noyau Laravel	4
5.2	Couche interface	5
5.3	Moteur d'intelligence artificielle	5
6	Etapes de fonctionnement du système	5
6.1	Etape 1 : saisie du formulaire	5
6.2	Etape 2 : validation Laravel	5
6.3	Etape 3 : appel HTTP vers FastAPI	5
6.4	Etape 4 : prédiction ML	5
6.5	Etape 5 : persistance	5
6.6	Etape 6 : statistiques	5
7	Backend Laravel	6
7.1	Routes web	6
7.2	Contrôleur de prédiction	6
7.3	Historique des prédictions	7
7.4	Gestion des offres d'emploi	7
7.5	Modèle Eloquent Prediction	7
7.6	Migration de la table predictions	8
8	Frontend Blade et Tailwind CSS	8
8.1	Vue d'accueil et formulaire de prédiction	8
8.2	Page de résultat	9
8.3	Tableau de bord statistique	9
8.4	Vue offres d'emploi	9
9	FastAPI et modèle de prédiction	10
9.1	Chargement du modèle et des données	10
9.2	Schéma de requête	10
9.3	Validation et normalisation	10
9.4	Endpoint principal de prédiction	11
9.5	Fonction d'inférence	11
10	Script d'entraînement du modèle	11
10.1	Construction du pipeline ML	11
10.2	Entraînement et sauvegarde	12
11	Contrat d'échange entre Laravel et FastAPI	12
11.1	Requête envoyée	13
11.2	Réponse retournée	13
12	Synthèse technique par couche	13

13 Points forts du projet	13
14 Conclusion	13

1 Présentation générale

JobAI est une plateforme web qui combine deux objectifs :

- estimer un salaire à partir d'un profil candidat ;
- publier et consulter des offres d'emploi.

L'application repose sur une séparation claire des responsabilités :

- **Laravel** gère l'interface, les routes web, la validation côté serveur et la base de données ;
- **Blade + Tailwind CSS** assurent le rendu frontend moderne ;
- **FastAPI** expose le service de prédiction ;
- **scikit-learn** fournit le modèle de machine learning utilisé pour estimer le salaire.

2 Objectifs fonctionnels

1. Permettre à un utilisateur de renseigner son expérience, son diplôme, son poste, sa ville et sa compétence principale.
2. Envoyer ces données au microservice FastAPI.
3. Recevoir une prédiction salariale.
4. Sauvegarder la prédiction dans la table `predictions`.
5. Afficher des statistiques agrégées par poste.
6. Gérer une mini-bourse d'offres d'emploi via Laravel.

3 Diagramme d'architecture

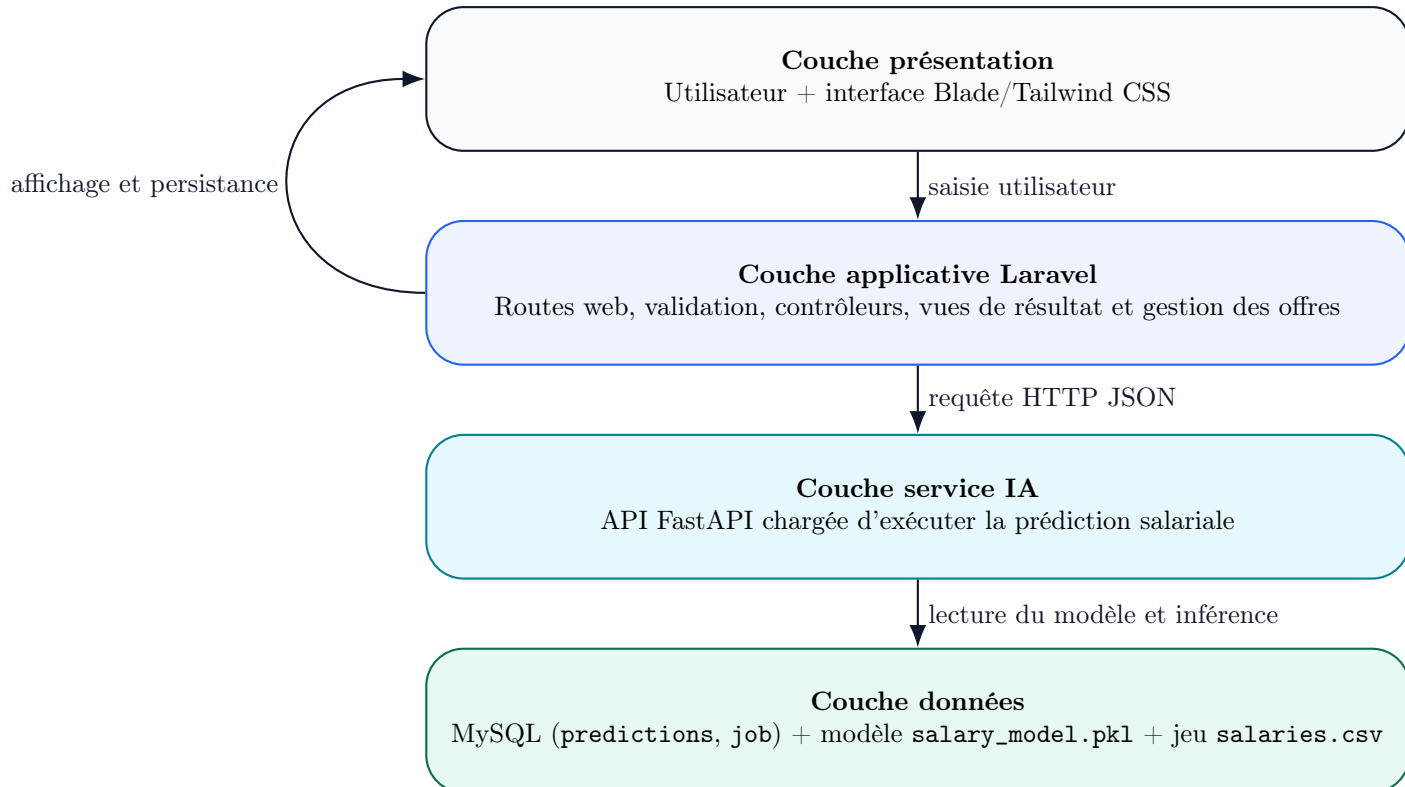


FIGURE 1 – Architecture logique en couches du projet JobAI

3.1 Lecture du flux

1. L'utilisateur remplit le formulaire de prédiction sur l'interface Blade.

2. Laravel valide les champs.
3. Laravel appelle FastAPI via HTTP.
4. FastAPI charge le modèle `salary_model.pkl` et calcule la prédiction.
5. Laravel enregistre la valeur prédite en base.
6. La vue `result.blade.php` affiche le salaire estimé.

4 Tableau des commandes

Bloc	Commande	Rôle
Laravel	<code>php artisan serve</code>	Démarre le serveur web Laravel local.
Laravel	<code>php artisan migrate</code>	Crée les tables de base de données, dont <code>predictions</code> .
Laravel	<code>php artisan route:list</code>	Affiche les routes disponibles du projet.
Laravel	<code>npm install</code>	Installe les dépendances frontend si Vite est utilisé.
Laravel	<code>npm run dev</code>	Lance le bundler frontend en mode développement.
FastAPI	<code>pip install -r requirements.txt</code>	Installe FastAPI, pandas, scikit-learn et les bibliothèques Python.
FastAPI	<code>python train_model.py</code>	Entraîne le modèle de prédiction puis génère <code>salary_model.pkl</code> .
FastAPI	<code>uvicorn app:app --reload --port 8000</code>	Démarre l'API FastAPI pour la prédiction.
Base de données	<code>php artisan tinker</code>	Permet de tester les modèles Eloquent rapidement.
Test API	<code>curl -X POST http://127.0.0.1:8000/predictors/salary</code> ...	Vérifie qu'une prédiction JSON est bien retournée.

5 Structure logique du projet

La structure du projet peut être lue par blocs fonctionnels plutôt que sous forme de tableau, ce qui rend la logique d'ensemble plus naturelle à suivre.

5.1 Noyau Laravel

- `plateforme/routes/web.php` centralise les routes web de l'application.
- `plateforme/app/Http/Controllers/PredictionController.php` orchestre la prédiction, l'enregistrement et l'historique.
- `plateforme/app/Http/Controllers/JobController.php` pilote les actions liées aux offres d'emploi.
- `plateforme/app/Models/Prediction.php` représente les prédictions stockées en base.
- `plateforme/app/Models/Job.php` modélise les offres publiées dans la plateforme.

5.2 Couche interface

- `plateforme/resources/views/welcome.blade.php` contient le formulaire principal de saisie.
- `plateforme/resources/views/result.blade.php` affiche le salaire calculé.
- `plateforme/resources/views/history.blade.php` présente l'historique et les graphiques statistiques.
- `plateforme/resources/views/jobs.blade.php` liste les opportunités disponibles.
- `plateforme/resources/views/admin/create-job.blade.php` permet l'ajout d'une nouvelle offre.

5.3 Moteur d'intelligence artificielle

- `fastapi2/app.py` expose le service FastAPI de prédiction.
- `fastapi2/train_model.py` entraîne le modèle de machine learning.
- `fastapi2/data/salaries.csv` fournit les données utilisées pour l'apprentissage.
- `fastapi2/salary_model.pkl` correspond au modèle final chargé lors des inférences.

6 Etapes de fonctionnement du système

6.1 Etape 1 : saisie du formulaire

L'utilisateur renseigne les champs suivants :

- années d'expérience ;
- diplôme ;
- ville ;
- poste ciblé ;
- compétence principale.

6.2 Etape 2 : validation Laravel

Le contrôleur Laravel s'assure que les champs sont présents et correctement typés avant de contacter le moteur de prédiction.

6.3 Etape 3 : appel HTTP vers FastAPI

Laravel envoie un payload JSON au point d'entrée `/predict-salary`.

6.4 Etape 4 : prédiction ML

FastAPI normalise les données, construit un `DataFrame`, appelle `model.predict()` et renvoie le salaire estimé.

6.5 Etape 5 : persistance

Le salaire estimé est enregistré dans la table `predictions` pour alimenter l'historique.

6.6 Etape 6 : statistiques

Laravel agrège les prédictions par `job_title` et affiche la moyenne par poste.

7 Backend Laravel

7.1 Routes web

Le fichier de routes centralise la navigation entre la page d'accueil, la prédiction, l'historique et les offres d'emploi.

```
1 use App\Http\Controllers\PredictionController;
2 use App\Http\Controllers\JobController;
3
4 Route::get('/', [PredictionController::class, 'index']);
5 Route::post('/predict', [PredictionController::class, 'predict']);
6 Route::get('/history', [PredictionController::class, 'history']);
7
8 Route::get('/jobs', [JobController::class, 'index']);
9 Route::get('/admin/jobs/create', [JobController::class, 'create']);
10 Route::post('/admin/jobs/store', [JobController::class, 'store']);
```

Listing 1 – Extrait de routes Laravel

Explication : ces routes lient directement les URL aux contrôleurs. Elles constituent la colonne vertébrale du backend Laravel.

7.2 Contrôleur de prédiction

Ce contrôleur valide la requête, appelle FastAPI et sauvegarde le résultat.

```
1 public function predict(Request $request)
2 {
3     $request->validate([
4         'experience_years' => 'required|numeric',
5         'degree' => 'required',
6         'job_title' => 'required',
7         'city' => 'required',
8         'skill' => 'required'
9     ]);
10
11     $response = Http::post('http://127.0.0.1:8000/predict-salary', [
12         'experience_years' => (float) $request->experience_years,
13         'degree' => $request->degree,
14         'job_title' => $request->job_title,
15         'city' => $request->city,
16         'skill' => $request->skill,
17     ]);
18
19     $salary = $response['predicted_salary'];
20
21     Prediction::create([
22         'user_id' => null,
23         'experience_years' => $request->experience_years,
24         'degree' => $request->degree,
25         'job_title' => $request->job_title,
26         'city' => $request->city,
27         'skill' => $request->skill,
28         'predicted_salary' => $salary,
29     ]);
30
31     return view('result', compact('salary'));
32 }
```

Listing 2 – Extrait important de PredictionController.php

Ce que fait ce code :

- applique une validation serveur ;
- appelle le microservice Python ;
- extrait la valeur `predicted_salary` ;
- persiste les données dans la base ;
- affiche la page de résultat.

7.3 Historique des prédictions

L'historique exploite Eloquent pour calculer une moyenne salariale par métier.

```

1 $predictions = Prediction::select('job_title')
2   ->selectRaw('AVG(predicted_salary) as avg_salary')
3   ->groupBy('job_title')
4   ->get();
5
6 $labels = $predictions->pluck('job_title');
7 $salaries = $predictions->pluck('avg_salary');
8
9 return view('history', compact('predictions', 'labels', 'salaries'));
```

Listing 3 – Agrégation des statistiques

Intérêt : ce bloc transforme les prédictions stockées en indicateurs exploitables pour le tableau de bord.

7.4 Gestion des offres d'emploi

Le contrôleur suivant permet d'ajouter de nouvelles offres dans la table job.

```

1 public function store(Request $request)
2 {
3     $request->validate([
4         'title' => 'required',
5         'company' => 'required',
6         'city' => 'required',
7         'email' => 'required|email',
8         'description' => 'required'
9     ]);
10
11     Job::create($request->all());
12
13     return redirect('/jobs')->with('success', 'Job ajoute avec succes');
14 }
```

Listing 4 – Extrait de JobController.php

7.5 Modèle Eloquent Prediction

```

1 class Prediction extends Model
2 {
3     protected $fillable = [
4         'user_id',
5         'experience_years',
6         'degree',
7         'job_title',
8         'city',
9         'skill',
```



```

10         'predicted_salary'
11     ];
12 }

```

Listing 5 – Modele Prediction

Explication : la propriété `$fillable` autorise l'insertion de masse en sécurisant les colonnes autorisées.

7.6 Migration de la table predictions

```

1 Schema::create('predictions', function (Blueprint $table) {
2     $table->id();
3     $table->unsignedBigInteger('user_id')->nullable();
4     $table->float('experience_years');
5     $table->string('degree');
6     $table->string('job_title');
7     $table->string('city');
8     $table->string('skill');
9     $table->float('predicted_salary');
10    $table->timestamps();
11 });

```

Listing 6 – Migration importante pour le stockage des prédictions

8 Frontend Blade et Tailwind CSS

8.1 Vue d'accueil et formulaire de prédiction

La vue `welcome.blade.php` sert de page d'atterrissage et de formulaire principal. Elle combine :

- une navigation claire;
- une mise en page responsive;
- un design premium avec dégradés, cartes et boutons modernes;
- un formulaire structuré autour de champs `select` et `input`.

```

1 <form method="POST" action="/predict" class="space-y-4">
2     @csrf
3
4     <div>
5         <label for="experience_years"
6             class="mb-2 block text-sm font-semibold text-slate-700">
7             Annees d'experience
8         </label>
9         <input
10            id="experience_years"
11            type="number"
12            step="0.1"
13            name="experience_years"
14            class="w-full rounded-2xl border border-slate-200 bg-slate-50
15                px-4 py-3 outline-none transition
16                focus:border-cyan-500 focus:bg-white
17                focus:ring-4 focus:ring-cyan-100"
18            required
19        >
20     </div>
21
22     <button type="submit"
23         class="w-full rounded-2xl bg-slate-950 px-6 py-4

```

```

24         text-sm font-semibold text-white transition hover:bg-cyan-700
           ">
25         Lancer la prediction
26     </button>
27 </form>

```

Listing 7 – Extrait Blade/Tailwind du formulaire principal

Pourquoi ce code est important :

- @csrf protège la requête contre les attaques CSRF ;
- les classes Tailwind accélèrent le design responsive ;
- le formulaire envoie directement les données à Laravel.

8.2 Page de résultat

La vue `result.blade.php` valorise la prédiction obtenue et met en avant la somme estimée.

```

1 <p class="mt-4 text-5xl font-black text-slate-950 md:text-6xl">
2     {{ number_format($salary, 0, ',', ' ') }}
3     <span class="text-2xl font-bold text-emerald-700 md:text-3xl">DT</span>
4 </p>

```

Listing 8 – Bloc d’affichage du salaire estime

Apport UX : le résultat est lisible, central, bien hiérarchisé et cohérent avec une interface premium.

8.3 Tableau de bord statistique

La vue `history.blade.php` utilise Chart.js pour transformer des données PHP en graphique interactif.

```

1 const labels = @json($labels);
2 const data = @json($salaries);
3
4 new Chart(ctx, {
5     type: 'bar',
6     data: {
7         labels: labels,
8         datasets: [{
9             label: 'Salaire moyen (DT)',
10            data: data
11        }]
12    }
13 });

```

Listing 9 – Injection des données PHP vers Chart.js

Idée clé : Laravel prépare les données, Blade les injecte dans le JavaScript, puis Chart.js les visualise.

8.4 Vue offres d’emploi

La vue `jobs.blade.php` affiche les annonces sous forme de cartes modernes, tandis que `create-job.blade.php` permet l’ajout d’une nouvelle offre.

```

1 @foreach($jobs as $job)
2     <article class="group flex h-full flex-col rounded-[2rem] border
3         border-white/70 bg-white p-6 shadow-sm">
4         <h2 class="mt-1 text-xl font-bold text-slate-950">{{ $job->title }}<
          /h2>
5         <p class="mt-1 text-sm text-slate-500">{{ $job->company }}</p>

```

```

6         <span class="rounded-full bg-slate-100 px-3 py-1
7             text-xs font-semibold text-slate-700">{{ $job->city }}<
            /span>
8     </article>
9 @endforeach

```

Listing 10 – Extrait Blade d’une carte offre d’emploi

9 FastAPI et modèle de prédiction

9.1 Chargement du modèle et des données

Le microservice FastAPI charge au démarrage le modèle entraîné et le fichier CSV contenant les valeurs de référence.

```

1 MODEL_PATH = Path("salary_model.pkl")
2 DATA_PATH = Path("data/salaries.csv")
3
4 with open(MODEL_PATH, "rb") as f:
5     model = pickle.load(f)
6
7 dataset = pd.read_csv(DATA_PATH)
8
9 FORM_OPTIONS = {
10     "degree": sorted(dataset["degree"].dropna().astype(str).unique().tolist()),
11     "job_title": sorted(dataset["job_title"].dropna().astype(str).unique().tolist()),
12     "city": sorted(dataset["city"].dropna().astype(str).unique().tolist()),
13     "skill": sorted(dataset["skill"].dropna().astype(str).unique().tolist())
14 }

```

Listing 11 – Chargement du modèle et des options de formulaire

Rôle : ce bloc rend l’API capable de valider les valeurs attendues et d’offrir une cohérence métier entre les données d’entraînement et l’inférence.

9.2 Schéma de requête

```

1 class SalaryPredictionRequest(BaseModel):
2     experience_years: float
3     degree: str
4     job_title: str
5     city: str
6     skill: str

```

Listing 12 – Schema Pydantic pour la prédiction

Explication : Pydantic structure les entrées, simplifie la validation et documente implicitement l’API.

9.3 Validation et normalisation

```

1 def normalize(payload: dict):
2     return {
3         "experience_years": payload["experience_years"],
4         "degree": payload["degree"].strip(),
5         "job_title": payload["job_title"].strip(),

```

```

6         "city": payload["city"].strip(),
7         "skill": payload["skill"].strip(),
8     }
9
10 def validate(payload: dict):
11     errors = []
12
13     if payload["experience_years"] < 0 or payload["experience_years"] > 50:
14         errors.append("experience_years must be between 0 and 50")
15
16     for f in ["degree", "job_title", "city", "skill"]:
17         if not payload[f]:
18             errors.append(f"{f} is required")
19
20     for f in ["degree", "job_title", "city", "skill"]:
21         if payload[f] not in FORM_OPTIONS[f]:
22             errors.append(f"{f} invalid value")
23
24     return errors

```

Listing 13 – Fonctions de preparation des donnees

Intérêt technique :

- nettoyage des chaînes;
- contrôle des bornes sur l'expérience;
- vérification des valeurs autorisées.

9.4 Endpoint principal de prédiction

```

1 @app.post("/predict-salary")
2 def predict_simple(data: SalaryPredictionRequest):
3     payload = normalize(data.model_dump())
4     prediction = predict_salary(payload)
5
6     return {
7         "predicted_salary": prediction
8     }

```

Listing 14 – Endpoint FastAPI pour Laravel

Lecture : Laravel appelle cet endpoint, reçoit un JSON simple et peut afficher le salaire sans transformation complexe.

9.5 Fonction d'inférence

```

1 def predict_salary(payload: dict):
2     df = pd.DataFrame([payload])
3     pred = model.predict(df)[0]
4     return round(float(pred), 2)

```

Listing 15 – Inference scikit-learn

Point clé : la prédiction repose sur une seule ligne de données transformée en `DataFrame` pour rester compatible avec le pipeline scikit-learn.

10 Script d'entraînement du modèle

10.1 Construction du pipeline ML

Le script `train_model.py` crée un pipeline comprenant :

- `OneHotEncoder` pour les variables catégorielles ;
- `passthrough` pour la variable numérique `experience_years` ;
- `RandomForestRegressor` pour la régression finale.

```

1 def build_pipeline() -> Pipeline:
2     categorical_features = ["degree", "job_title", "city", "skill"]
3     numeric_features = ["experience_years"]
4
5     preprocessor = ColumnTransformer(
6         transformers=[
7             ("cat", OneHotEncoder(handle_unknown="ignore"),
8               categorical_features),
9             ("num", "passthrough", numeric_features),
10            ]
11        )
12
13     model = RandomForestRegressor(
14         n_estimators=300,
15         max_depth=12,
16         min_samples_split=2,
17         min_samples_leaf=1,
18         random_state=42,
19     )
20
21     return Pipeline([("preprocessor", preprocessor), ("model", model)])

```

Listing 16 – Pipeline de machine learning

10.2 Entraînement et sauvegarde

```

1 df = load_dataset()
2 X = df[FEATURES]
3 y = df["salary"]
4 pipeline = build_pipeline()
5
6 X_train, X_test, y_train, y_test = train_test_split(
7     X, y, test_size=0.2, random_state=42
8 )
9
10 pipeline.fit(X_train, y_train)
11 predictions = pipeline.predict(X_test)
12
13 mae = mean_absolute_error(y_test, predictions)
14 r2 = r2_score(y_test, predictions)
15
16 with open(MODEL_PATH, "wb") as file_handle:
17     pickle.dump(pipeline, file_handle)

```

Listing 17 – Apprentissage, évaluation et export du modèle

Analyse :

- MAE mesure l'erreur absolue moyenne ;
- R2 mesure la qualité d'ajustement du modèle ;
- le pipeline entier est sérialisé dans `salary_model.pkl`.

11 Contrat d'échange entre Laravel et FastAPI

11.1 Requête envoyée

```

1 {
2   "experience_years": 3.5,
3   "degree": "Master",
4   "job_title": "Data Scientist",
5   "city": "Tunis",
6   "skill": "Python"
7 }
```

Listing 18 – Exemple de payload JSON

11.2 Réponse retournée

```

1 {
2   "predicted_salary": 3200.75
3 }
```

Listing 19 – Reponse JSON simplifiée

12 Synthèse technique par couche

Couche	Technologie	Responsabilité
Interface	Blade + Tailwind CSS	Affichage, formulaire, UX premium, responsive design.
Contrôle métier	Laravel Controllers	Validation, orchestration du flux, persistance.
Persistance	Eloquent + migrations	Stockage des prédictions et des offres.
Visualisation	Chart.js	Représentation graphique des moyennes salariales.
IA / API	FastAPI + Pydantic	Exposition du service de prédiction.
Machine Learning	pandas + scikit-learn	Entraînement, encodage, prédiction du salaire.

13 Points forts du projet

- architecture simple à comprendre et facile à démontrer ;
- séparation nette entre interface web et moteur de prédiction ;
- exploitation d'un vrai pipeline machine learning ;
- design frontend moderne avec Tailwind CSS ;
- exploitation des prédictions stockées pour produire des statistiques.

14 Conclusion

Le projet JobAI illustre une architecture hybride très pertinente pour un projet académique ou professionnel : Laravel pilote l'expérience utilisateur et la logique web, tandis que FastAPI fournit une brique IA spécialisée pour la prédiction de salaire. Le frontend Blade/Tailwind CSS apporte une

présentation moderne, et la couche machine learning permet de valoriser les données par une estimation concrète et exploitable.

Ce document LaTeX regroupe les éléments les plus importants du projet : page de garde premium, diagramme d'architecture, tableau des commandes, extraits de code essentiels et explications claires des différentes étapes.